

Insertion Sort

Insertion Sort 的原理是將元素逐一插入到已排序區域中的正確位置。此演算法結構簡單，而且在小型資料集上具有不錯的效率。

然而，當輸入資料為反向排序 (descending order) 時，需要進行大量比較與位移，因此會形成 worst-case condition。隨著資料量增加，其執行時間會快速上升。

雖然 Insertion Sort 不適合大型資料集，但由於其 overhead 較低，因此仍然常被應用於 hybrid sorting 中的小型 partition

```
#include <iostream>
#include <vector>

using namespace std;

// Insertion Sort Function
void insertionSort(vector<int>& arr)
{
    int n = arr.size();

    // Traverse through the array
    for (int i = 1; i < n; i++)
    {
        // Current element to insert
        int key = arr[i];

        // Compare with previous elements
        int j = i - 1;

        // Shift elements greater than key
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        // Insert key into correct position
        arr[j + 1] = key;
    }
}

int main()
{
    vector<int> arr = {9, 5, 1, 4, 3};

    insertionSort(arr);
```

```
    cout << "Sorted Array: ";

    for (int num : arr)
        cout << num << " ";

    return 0;
}
```

Quick Sort (Median-of-Three)

Quick Sort 屬於 divide-and-conquer (分治法) 演算法，透過選擇 pivot 將資料切分成左右兩部分，再進行遞迴排序。

在本次專案中，我使用了 Median-of-Three 方法來改善 pivot selection。此方法會從 left、middle 與 right 三個元素中選出中位數作為 pivot，以降低不平衡 partition 發生的可能性。

Quick Sort 通常具有非常好的平均效能，因此在實務上常被視為速度最快的排序演算法之一。然而，若 partition 極度不平衡，仍然可能產生 $O(n^2)$ 的 worst-case complexity。

因此，在大型系統中通常會搭配其他演算法一起使用，以避免 recursion depth 過深

```
#include <iostream>
#include <vector>

using namespace std;

// Select pivot using Median-of-Three
int medianOfThree(vector<int>& arr, int left, int right)
{
    int mid = (left + right) / 2;

    // Sort left, middle, right
    if (arr[left] > arr[mid])
        swap(arr[left], arr[mid]);

    if (arr[left] > arr[right])
        swap(arr[left], arr[right]);

    if (arr[mid] > arr[right])
        swap(arr[mid], arr[right]);

    // Place pivot at right - 1
    swap(arr[mid], arr[right - 1]);

    return arr[right - 1];
}
```

```

// Recursive Quick Sort helper
void quickSortHelper(vector<int>& arr, int left, int right)
{
    // Use Quick Sort only for larger partitions
    if (left + 10 <= right)
    {
        int pivot = medianOfThree(arr, left, right);

        int i = left;
        int j = right - 1;

        while (true)
        {
            // Move left pointer
            while (arr[++i] < pivot) {}

            // Move right pointer
            while (arr[--j] > pivot) {}

            // Swap elements
            if (i < j)
                swap(arr[i], arr[j]);
            else
                break;
        }

        // Restore pivot
        swap(arr[i], arr[right - 1]);

        // Recursive calls
        quickSortHelper(arr, left, i - 1);
        quickSortHelper(arr, i + 1, right);
    }
}

// Main Quick Sort function
void quickSort(vector<int>& arr)
{
    quickSortHelper(arr, 0, arr.size() - 1);
}

int main()
{
    vector<int> arr = {9, 5, 1, 4, 3};

    quickSort(arr);

    cout << "Sorted Array: ";
}

```

```
    for (int num : arr)
        cout << num << " ";

    return 0;
}
```

Iterative Merge Sort

Merge Sort 的核心概念是將陣列持續切分成較小的 subarrays, 再透過 merge operation 將它們重新合併成排序完成的結果。

本次作業中, 我使用 iterative Merge Sort 來降低 recursion overhead。

Merge Sort 最大的優點是無論資料排列方式如何, 其時間複雜度都能穩定維持在 $O(n \log n)$, 因此非常適合大型資料排序。

此外, Merge Sort 屬於 stable sorting algorithm, 也就是當元素相同時, 可以保留原本的相對順序。這在資料庫排序或多層排序中非常重要。

不過, 其缺點是需要額外 temporary arrays, 因此 memory usage 較高

```
#include <iostream>
#include <vector>

using namespace std;

// Merge two sorted subarrays
void merge(vector<int>& arr, int left, int mid, int right)
{
    vector<int> temp(right - left + 1);

    int i = left;
    int j = mid + 1;
    int k = 0;

    // Compare and merge elements
    while (i <= mid && j <= right)
    {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    // Copy remaining left elements
    while (i <= mid)
        temp[k++] = arr[i++];
}
```

```

// Copy remaining right elements
while (j <= right)
    temp[k++] = arr[j++];

// Copy back to original array
for (int p = 0; p < temp.size(); p++)
    arr[left + p] = temp[p];
}

// Iterative Merge Sort
void iterativeMergeSort(vector<int>& arr)
{
    int n = arr.size();

    // Current subarray size
    for (int currSize = 1; currSize < n; currSize *= 2)
    {
        // Merge subarrays
        for (int left = 0; left < n - 1; left += 2 * currSize)
        {
            int mid = min(left + currSize - 1, n - 1);
            int right = min(left + 2 * currSize - 1, n - 1);

            if (mid < right)
                merge(arr, left, mid, right);
        }
    }
}

int main()
{
    vector<int> arr = {9, 5, 1, 4, 3};

    iterativeMergeSort(arr);

    cout << "Sorted Array: ";

    for (int num : arr)
        cout << num << " ";

    return 0;
}

```

Heap Sort

Heap Sort 會先將資料建立成 binary heap, 再不斷取出最大元素完成排序。

Heap Sort 的優點是可以保證 $O(n \log n)$ worst-case complexity, 而且不需要像 Merge Sort 一樣使用大量額外記憶體。

此外, Heap Sort 對 worst-case input 的敏感度較低, 因此在需要穩定 worst-case performance 的系統中特別重要。

然而, 由於 heap operations 涉及較多 memory jumps, 因此在實際執行速度上通常會略慢於 Quick Sort

```
#include <iostream>
#include <vector>

using namespace std;

// Heapify subtree
void heapify(vector<int>& arr, int n, int i)
{
    int largest = i;

    // Left child index
    int left = 2 * i + 1;

    // Right child index
    int right = 2 * i + 2;

    // Compare left child
    if (left < n && arr[left] > arr[largest])
        largest = left;

    // Compare right child
    if (right < n && arr[right] > arr[largest])
        largest = right;

    // Swap and continue heapifying
    if (largest != i)
    {
        swap(arr[i], arr[largest]);

        heapify(arr, n, largest);
    }
}

// Heap Sort function
void heapSort(vector<int>& arr)
{
    int n = arr.size();

    // Build max heap
```

```

    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Extract elements from heap
    for (int i = n - 1; i > 0; i--)
    {
        // Move largest element to end
        swap(arr[0], arr[i]);

        // Heapify reduced heap
        heapify(arr, i, 0);
    }
}

int main()
{
    vector<int> arr = {9, 5, 1, 4, 3};

    heapSort(arr);

    cout << "Sorted Array: ";

    for (int num : arr)
        cout << num << " ";

    return 0;
}

```

實驗結果顯示，Insertion Sort 僅適合小型資料集。當資料量增加時，由於 $O(n^2)$ complexity，其執行時間明顯快速上升。

Quick Sort 在大部分情況下具有最佳 average performance，而 Median-of-Three 技術也有效降低了不平衡 partition 的問題。

Merge Sort 與 Heap Sort 則展現出更穩定的 runtime performance，特別是在大型資料集下，其 $O(n \log n)$ complexity 明顯優於 Insertion Sort。

其中，Merge Sort 的 runtime 最穩定，而 Heap Sort 則在 memory efficiency 與 worst-case guarantee 上具有優勢。